

Enlight Lab

Full Client Demo Script

Unified DevOps platform demo - browser UI only.

Share: <http://localhost:30900> (Demo Control Center)

Project: D:\enlight-lab-platform

1. BEFORE THE DEMO (15 min - hide from client)

```
cd D:\enlight-lab-platform
.\scripts\go-live.bat
.\start-demo-control.bat
```

Keep PowerShell minimized. Do NOT share terminal.

Quick checks

- <http://localhost:30900> - Demo Control opens
- Click Refresh - Dashboard / Health / IDP should be green
- <http://localhost:8082> - ArgoCD (NOT 8080 - that is k8sgpt)
- <http://localhost:30800/health> - shows {"status":"ok"}

2. URLS TO KNOW

What	URL
PRIMARY (share this)	http://localhost:30900
App dashboard	http://localhost:30800
Health check	http://localhost:30800/health
IDP portal	http://localhost:30800/idp
ArgoCD	http://localhost:8082
Grafana	http://localhost:3000 (admin / enlight-admin)
GitHub Actions	github.com/kirtiprasad2003/enlight-lab-platform/actions

3. PLATFORM STORY (30 sec opening)

Enlight Lab unifies CI policy, GitOps, monitoring, AI incident response, IDP golden-path scaffolding, drift detection, and PR compliance on one platform. Runs locally at zero cloud cost; same architecture on EKS in production.

4. GUIDED DEMO - 10 STEPS IN DEMO CONTROL

Open <http://localhost:30900>. Use the numbered guided steps OR the buttons below.

1. Check platform

Click Refresh. All status cards green.

2. Demo 2 BLOCK

Block bad deploy. Show GitHub Actions RED / policy-check failed.

3. Demo 2 PASS

Pass good deploy. Show GitHub Actions GREEN.

4. Demo 1 Inject

Inject failure. ArgoCD fastapi-staging goes Degraded.

5. Demo 1 Explain

AI explain. Show k8sgpt findings in Live activity.

6. Demo 1 Heal

Heal / rollback. Then click Refresh (restores browser tunnel).

7. Demo 5 BLOCK

Block bad PR. New GitHub PR fails compliance checks.

8. Demo 3 Scaffold

Scaffold new. Creates NEW service svc-YYYYMMDDHHMMSS.

9. Demo 4 Drift

Detect drift. Demo 4 panel turns RED - DRIFT DETECTED.

10. Demo 4 Reconcile

Reconcile. Panel turns green - IN SYNC again.

5. DEMO 2 - POLICY GATE (CI blocks bad deploy)

Story: Bad manifest never reaches the cluster.

UI buttons

- Block bad deploy - dispatches chat-to-deploy non-compliant
- Pass good deploy - dispatches chat-to-deploy compliant

Show client

- Live activity - workflow result
- Open GitHub link - policy-check step RED (block) or GREEN (pass)

What policy catches

- Forbidden :latest image tag
- Unapproved registry
- Missing CPU/memory limits

6. DEMO 1 - INCIDENT RESPONSE

Story: App breaks -> AI explains -> GitOps heals.

Flow

1. Inject failure - bad image on fastapi (on purpose)
2. Show ArgoCD <http://localhost:8082> -> fastapi-staging RED
3. AI explain - k8sgpt findings in console
4. Heal / rollback - restores good image + selfHeal
5. Click Refresh - Dashboard/Health/IDP green again

Important

After Inject, Dashboard/Health/IDP may show FAIL while ArgoCD still shows Synced - that means port-forward died, not the cluster. Heal fixes the pod; Refresh fixes the browser tunnel.

Grafana (optional)

fastapi has no custom dashboard. Use Grafana K8s pod views in enlight-staging, or show ArgoCD as primary incident signal.

7. DEMO 5 - PR COMPLIANCE

Story: Every PR scanned - bad config cannot merge.

UI buttons

- Block bad PR - opens NEW GitHub PR with secrets + bad S3
- Pass good PR - opens NEW GitHub PR with clean config

Show client

- ONE PR, TWO workflow runs (service CI + PR Compliance Bot) - normal
- PR Checks tab - RED (block) or GREEN (pass)

Merge is optional for demo; checks are the proof.

8. DEMO 3 - IDP GOLDEN PATH (MAIN DEMO)

Story: Developer portal scaffolds a new service -> PR for review -> ArgoCD deploys with monitoring pre-wired. This is the platform vision demo.

What each click does

Button	Action
Scaffold new	Creates svc-TIMESTAMP with K8s, CI, Terraform, ArgoCD app
Create PR	Opens GitHub PR; 2 CI checks run on that PR
Deploy	Registers app in ArgoCD + applies manifests

Client flow (recommended)

1. Open IDP portal <http://localhost:30800/idp>
2. Demo Control -> Scaffold new (note svc- name in Current service)
3. Create PR -> open GitHub Checks link
4. Merge PR on GitHub (optional for full GitOps story)
5. Deploy -> show NEW app in ArgoCD <http://localhost:8082>

Key points for client

- Each scaffold = NEW app (not same demo-api every time)
- Every change is a reviewable PR - not manual kubectl
- Bundle includes catalog, K8s, ServiceMonitor, alerts, Grafana dashboard
- Local equivalent of Backstage - same story, not full Backstage install

9. DEMO 4 - DRIFT & COST SENTINEL

Story: Terraform is source of truth. Someone changes AWS in console -> drift detected -> cost exposure -> reconcile back to Git.

UI (all in Demo Control - no terminal)

1. Reset baseline - S3 private + encrypted (Floci local AWS)
2. Detect drift - simulates public S3 ACL; panel turns RED
3. Reconcile - Terraform apply; panel turns green IN SYNC

Show client

- Demo 4 drift panel - desired private vs actual public-read
- Live activity - DRIFT DETECTED / RECONCILED highlighted

NOT on GitHub - runs locally via Floci + Terraform.

10. TROUBLESHOOTING

Problem	Fix
Dashboard/Health/IDP fail	go-live.bat then Refresh in Demo Control
ArgoCD 8082 not loading	port-forward-all.ps1 (8080 is k8sgpt)
After Demo 1 Heal still fail	Click Refresh - tunnel not pod
Demo 3 ArgoCD path error	Merge PR first, or Deploy uses PR branch
Demo 5 both workflows	1 PR + 2 checks = normal, not 2 PRs
Cluster dead after reboot	go-live.bat (may take 10-15 min)

11. PRIVATE COMMANDS (if not using UI)

```
cd D:\enlight-lab-platform
.\scripts\go-live.bat
.\start-demo-control.bat
# Demo 2: use Demo Control buttons or GitHub dispatch
# Demo 1: Inject / Explain / Heal buttons
# Demo 3: Scaffold new -> Create PR -> Deploy
# Demo 4: Reset baseline -> Detect drift -> Reconcile
# Demo 5: Block bad PR -> Pass good PR
```

12. CLOSING

Five integrated demos on one platform. Demo 3 (IDP) is the main growth story; Demos 1, 2, and 5 prove safety gates; Demo 4 proves infra governance. Zero cloud cost today - production-ready architecture on EKS.

13. LIKELY QUESTIONS

Why not AWS?

Local kind = \$0. EKS Terraform scaffold ready for production.

Is Backstage installed?

IDP portal + scripts simulate Backstage golden path locally.

Why 2 GitHub workflows on one PR?

Service CI + org compliance bot - both gates on every PR.

What does AI do?

Explains incidents (k8sgpt). ArgoCD/GitOps performs rollback.